AI Powered Pothole Detection and Intelligent Route Optimization System

Submitted by

Zain Mahedi (1WA23CS056) Thanu George (1WA23CS019)

Vikas Shashi (1WA23CS043) Tanish Agrawal (1WA23CS008)

Under the guidance of

Dr. Seema Patil

Associate Professor, Department of CSE, BMSCE

Department of Computer Science and Engineering

B.M.S. College of Engineering

Outline

1. Sustainable Development Goals and Collaboration
2. Problem Statement and Objectives
3. Existing System and Proposed System
4. High Level Design
5. Detailed Design
6. Implementation
7. Results: Datasets, Tables, Graphs and Snapshots of the Output
8. Conclusion and Future Enhancements
9. Paper Published
10. References

Sustainable Development Goals Addressed

SDG 3
Good Health and Well-being

Target 3.6: fewer road injuries

Potholes are a recorded cause of road accidents in India. A forward-facing camera records a pothole before a vehicle reaches it, and the route planner can steer around known ones. We provide information; no effect on accidents was measured.

SDG 9
Industry, Innovation and Infrastructure

Target 9.1: resilient infrastructure

Ordinary phone or dashcam video plus GPS becomes mapped potholes on road segments, each with its surveyed length and potholes per 100 m, so inspections can be directed without a survey vehicle.

SDG 11
Sustainable Cities and Communities

Target 11.2: safe, sustainable transport

The route planner offers up to three routes, from shortest to strongest pothole avoidance, with the potholes passed and how much of each route has actually been surveyed.

Collaboration

Project category

Community oriented project. Potholes affect every road user in Bengaluru; the system turns ordinary phone video into a pothole map that residents, road authorities and the city can use.

Multidisciplinary project. It combines computer vision (deep-learning detection), GPS and geospatial processing, road-network routing on OpenStreetMap, and a web application with a database.

Field work

We recorded 13 road videos on Bengaluru roads (13 and 22 October 2026) from a two-wheeler and on foot, each with a time-synchronised GPS log from our Android recorder app.

The recordings were processed end to end: detections placed on the map along the recorded route, and the three model versions compared on the same frames.

Collaborating organisation: [to be added by the team]

Problem Statement

Detect potholes in ordinary road video recorded from a phone or dashcam, place them on the road map using GPS, and recommend routes that avoid them, with results that are measured honestly on data the model has never seen.

1 Detection is manual

Most potholes are found through citizen complaints and manual surveys, which are slow, occasional and rarely record the exact location.

2 Navigation ignores the road surface

Route planners choose by distance and traffic alone and tell drivers nothing about the condition of the road ahead.

3 Our Phase 1 prototype was not trustworthy

It was evaluated only on its own data split. On our own recordings it boxed whole street scenes as potholes in 3,107 frames, and its severity rules were never calibrated.

Objectives

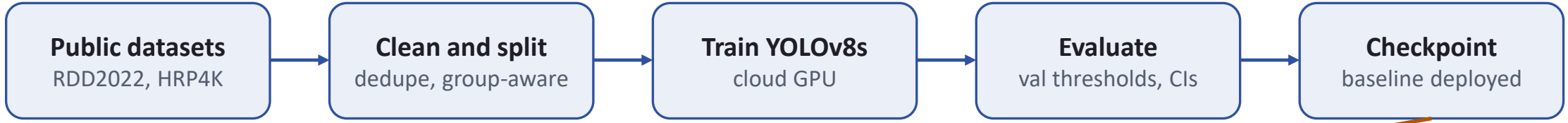
1. Prepare RDD2022 and HRP4K as one pothole class, with near-duplicate images grouped by perceptual hash and whole groups kept in one split, so no test image has a copy in training.
2. Train a reproducible YOLOv8s detection baseline with a fixed seed, recorded configuration and an archived, hashed checkpoint.
3. Evaluate with thresholds chosen on validation data only, 95% bootstrap confidence intervals and paired model comparisons.
4. Fine-tune the baseline on forward-view HRP4K footage at 1280 px and compare it with the baseline on held-out test sets.
5. Build the detection-to-map pipeline: video and GPS in; tracked, geotagged, road-matched potholes out.
6. Recommend pothole-aware routes through a tested backend (HTTP API, job queue, database) and a web interface.

Existing System and Proposed System

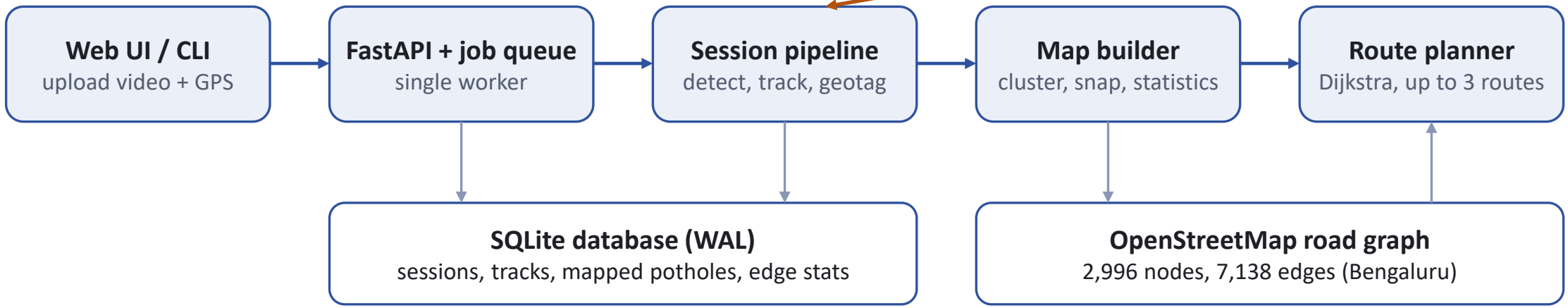
Aspect	Phase 1 prototype (existing)	Rebuilt system (proposed)
Model	YOLOv8s-seg masks, one pothole video dataset	YOLOv8s boxes, RDD2022 + HRP4K, single class
Data hygiene	Split by video only	Perceptual-hash near-duplicate groups, leakage guard
Evaluation	Scores on its own test split	Validation-fixed thresholds, bootstrap CIs, paired tests, external manhole set
Tracking	ByteTrack on every 5th frame	Motion-aware IoU tracker; every frame on vehicle drives
Road condition	Mild / Moderate / Severe rules, uncalibrated	Potholes per 100 m of surveyed road, per road segment
Routing	Public OSRM routes, potholes within 20 m	Own OpenStreetMap graph, Dijkstra with pothole cost; unsurveyed road treated as unknown
Engineering	FastAPI app, 50 Python tests	FastAPI + SQLite + job queue, 1,101 automated tests

High Level Design

Offline: model development

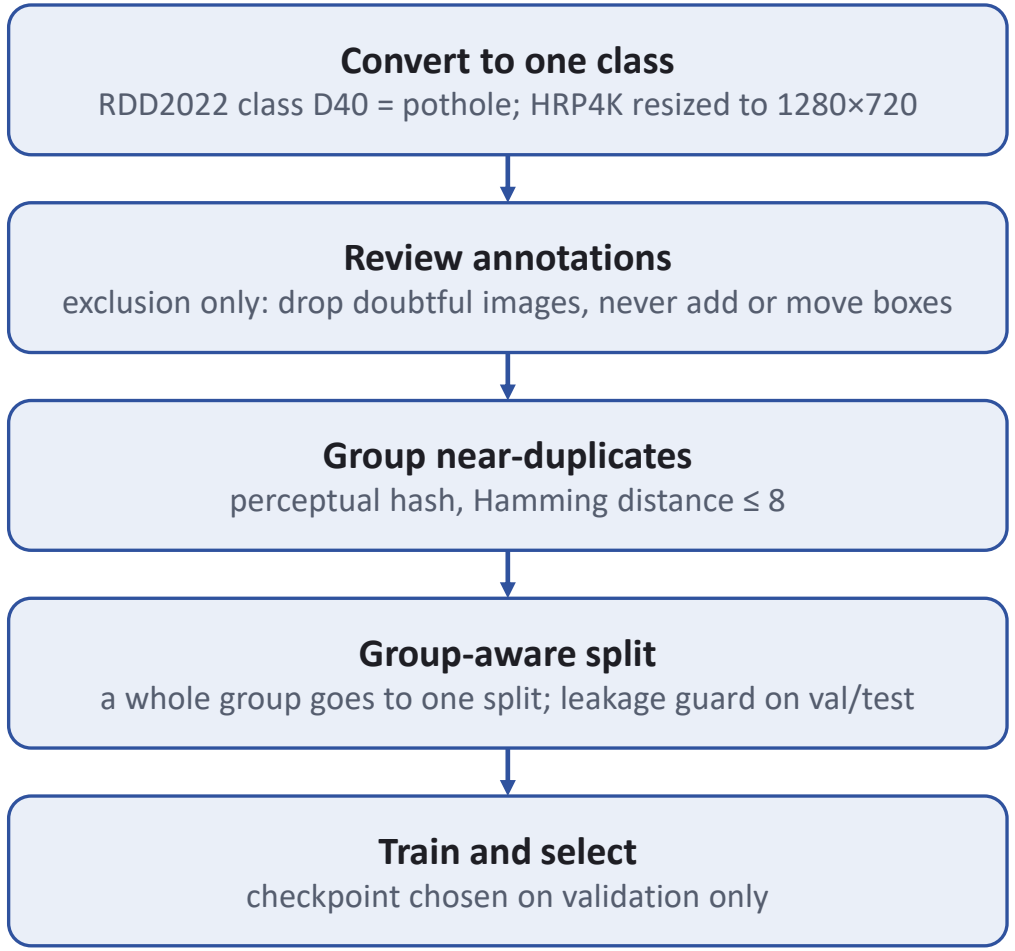


Online: the application



One process serves the API and the web interface. An uploaded recording becomes a job: frames are analysed, potholes are tracked and geotagged, the map is rebuilt, and the route planner reads the updated per-road statistics. The orange arrow is the deployed checkpoint feeding the detector.

Detailed Design: Data and Evaluation Pipeline



Evaluation protocol

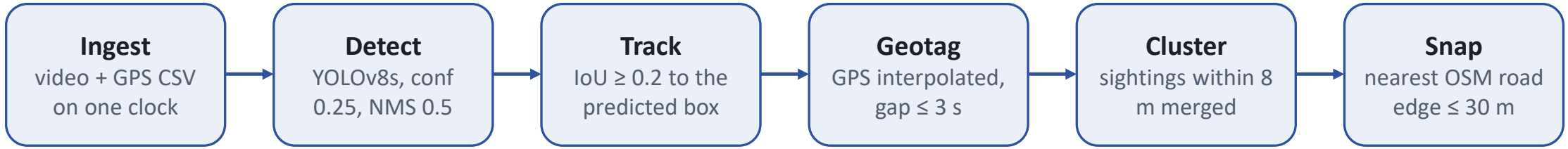
Confidence threshold set at maximum F1 on validation data, then applied unchanged to the held-out test sets.

Metrics per dataset: mAP50, mAP50-95, precision, recall, F1, false positives per image.

95% intervals from a group bootstrap (1,000 resamples); model differences paired on identical resamples.

An external 203-image set with intact manholes checks whether covers are mistaken for potholes.

Detailed Design: From Video to Mapped Potholes



Vehicle and walking recordings

A session whose upper-quartile GPS speed is at least 2.5 m/s is a vehicle drive: every frame is analysed and a track needs 6 detections to count as a pothole.

Slower sessions analyse every 3rd frame and need 3 detections. Short tracks are dropped as noise.

Road condition per segment

Each road edge stores the potholes mapped on it, the length actually surveyed (GPS trace ± 5 m) and the rate per 100 m of surveyed road.

Road that no video covered is shown as unknown, never as pothole-free. No severity or depth is estimated.

Detailed Design: Pothole-Aware Route Scoring

$$cost = L + a \times 300\ m \times E \qquad E = n + r \times (L - s) / 100$$

L edge length, a avoidance setting (0 to 1), n potholes seen on the edge, s surveyed length, r assumed rate on unsurveyed road

Three routes

Dijkstra's algorithm runs at avoidance 0, at the user's setting and at 1, giving the shortest route, the user's route and the strongest avoidance.

Unsurveyed road

Its rate is the observed rate blended with an assumed 1.0 pothole per 100 m (weight of 500 m of survey), so unknown road is never treated as free of potholes.

Evidence level

Each route reports the share that was surveyed: adequate from 80%, insufficient below 20%, partial in between, so users know how far to trust it.

Implementation: Technologies and Modules

Area	Used in this project
Language	Python 3.10
Deep learning	PyTorch 2.14.1, Ultralytics 8.4.210 (YOLOv8s)
Video	OpenCV
Backend	FastAPI 0.162.4, single-worker job queue
Storage	SQLite (WAL), 9 tables
Maps and routing	OSMnx 2.0.7, networkx 3.4.2, Leaflet
Training GPU	RTX 5090 (rented, Vast.ai)
Development GPU	RTX 4060 Laptop GPU

Ingest and detection

Video and GPS reader, YOLOv8s wrapper that refuses any non single-class checkpoint

Tracking and mapping

Motion-aware tracker, GPS interpolation, 8 m clustering, road snapping, per-edge statistics

Routing

OSM graph stored as JSON, Dijkstra with the pothole-aware cost, up to three routes

API and interface

20 HTTP endpoints; web UI with Dashboard, Upload, Sessions, Detections and Route planner tabs

Implementation: Training, Evaluation and Testing

	Baseline (deployed)	Candidate (experimental)
Starts from	YOLOv8s COCO weights	baseline checkpoint
Training data	RDD2022	RDD2022 + HRP4K
Input size	640 px	1280 px
Optimiser	MuSGD, lr 0.01	AdamW, lr 0.001, cosine
Epochs	76 of 100	50 (checkpoint chosen on validation)
Training run	RTX 5090	RTX 5090, 79 min, about $0.77

The cloud run was scripted end to end: upload of training and validation data only, training with a watchdog, checkpoint download, then the instance destroyed. Test data never left the laptop.

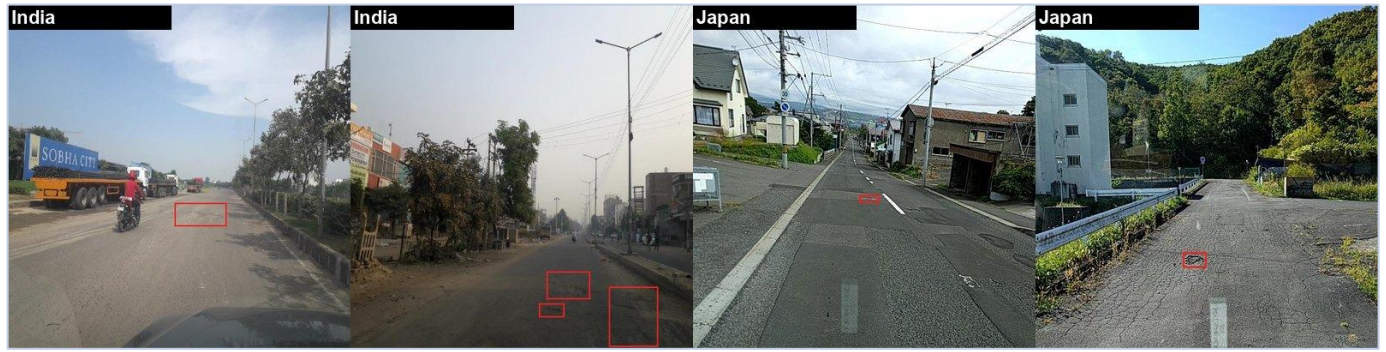
1,101
automated tests passing (pytest)

8.2 ms
per 640 px frame on the laptop GPU (baseline)

16.7 ms
per 1280 px frame (candidate), still real time

Results: Datasets Used

Source (images / boxes)	Train	Validation	Test
RDD2022 (India, Japan, Czech, US, China)	8,220 / 5,017	2,784 / 477	2,784 / 503
HRP4K (forward-view 4K frames)	3,973 / 4,709	825 / 922	811 / 836
Total	12,193 / 9,726	3,609 / 1,399	3,595 / 1,339

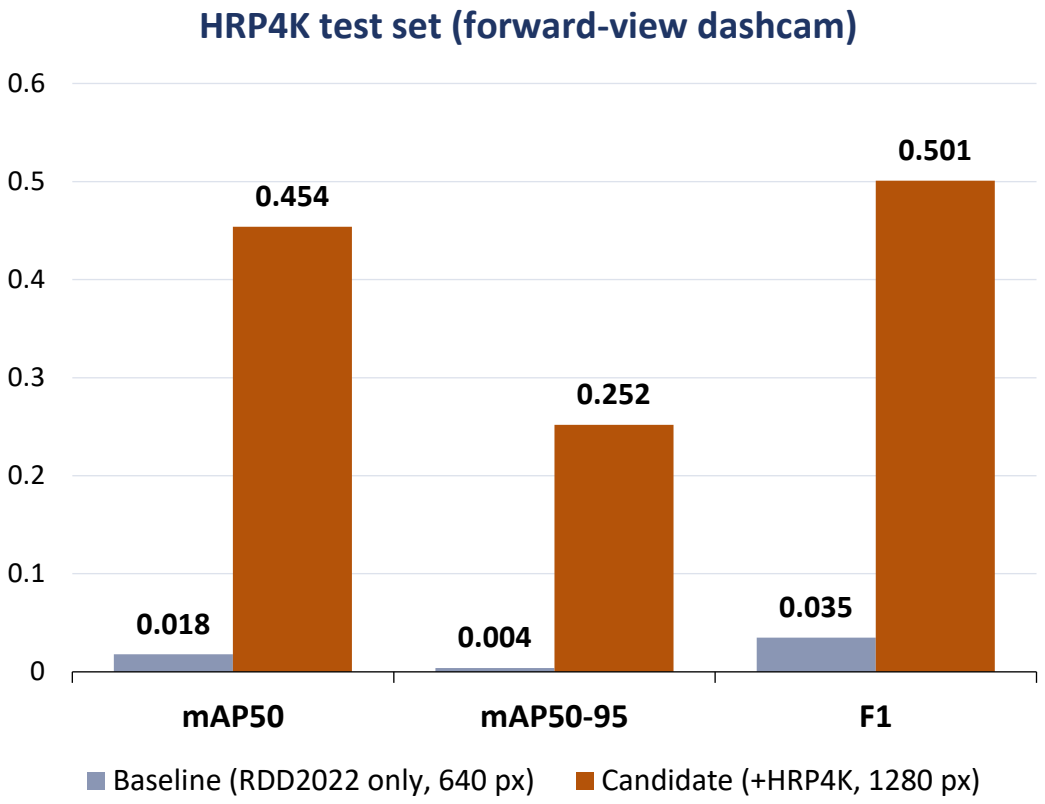


RDD2022 examples with pothole boxes (India, Japan).

Plus an evaluation-only set of 203 images (59 potholes, 196 intact manholes) to test manhole confusion.

Licences: RDD2022 CC BY-SA 4.0; HRP4K CC BY 4.0.

Results: Held-Out Test Performance



Test set (mAP50)	Baseline	Candidate	Difference [95% CI]
HRP4K	0.018	0.454	+0.435 [0.392, 0.479]
Manhole set	0.368	0.427	+0.058 [−0.027, 0.195]
RDD2022	0.497	0.453	−0.045 [−0.086, −0.006]

- Fine-tuning at 1280 px makes the detector work on forward-view footage: mAP50 rises 25 times on HRP4K.
- On RDD2022 it gives up some recall (−0.045 mAP50), and it does not reduce false positives on manholes.
- **So the baseline stays deployed and the 1280 px model stays experimental.**

Thresholds chosen on validation, applied unchanged to test.

Results: What We Tested on Validation Data

1 Threshold tuning

No threshold beat the pooled max-F1 threshold (0.24). F1 stays within 0.01 of its best over a 0.10 to 0.15 wide range. A higher threshold (0.37) halves false alarms per image (0.097 to 0.051) at the cost of recall.

2 Two-model ensemble

Weighted Boxes Fusion of the baseline (640 px) and the candidate (960 px) matched the baseline on RDD2022 but scored below the candidate alone on HRP4K (mAP50 −0.044). No overall gain, at twice the compute.

3 False-positive review

Of 260 candidate false positives on HRP4K validation, about a fifth were boxes of the wrong size on real potholes and about 38% were tiny boxes. Manholes and drains were rare.

All choices here were made on validation data only; the held-out test sets were never used for tuning.

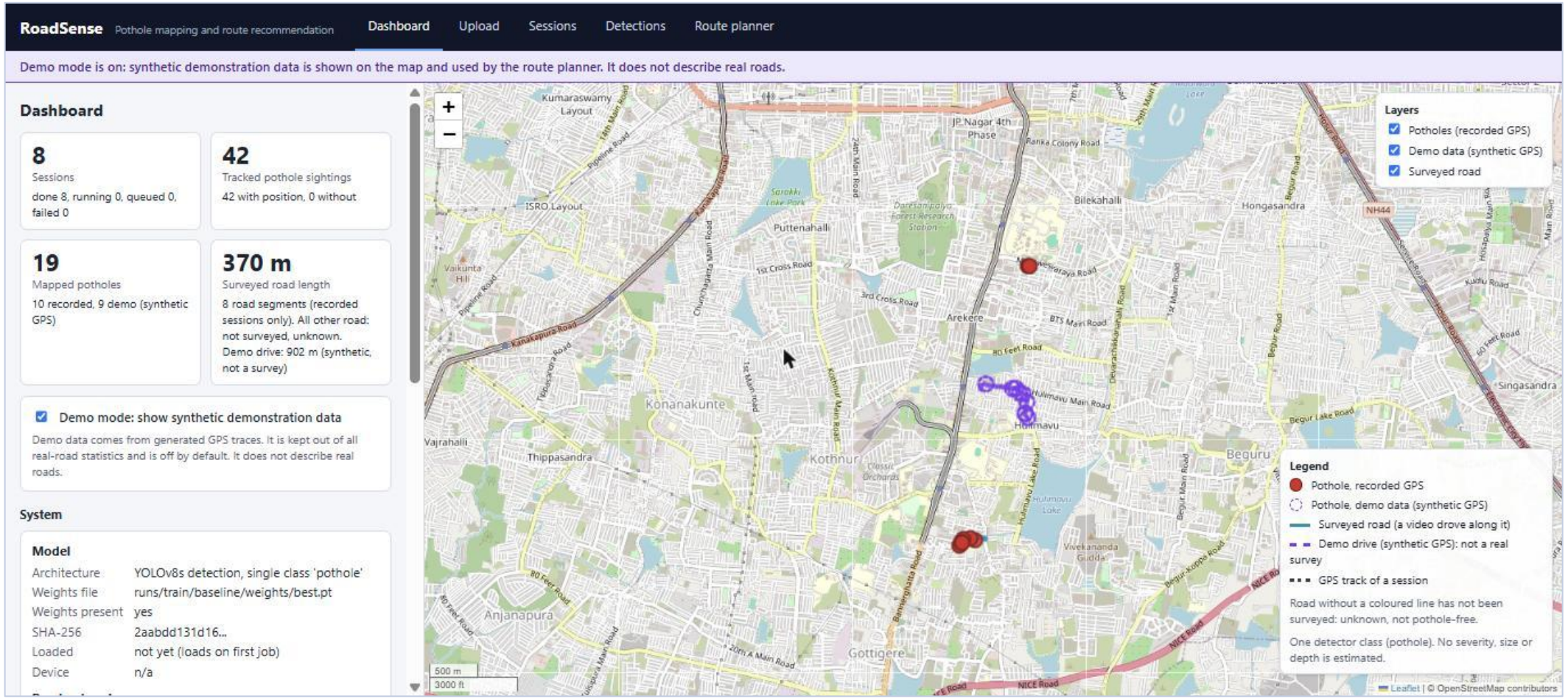
Results: Field Test on Bengaluru Roads

13 recordings, 9,711 frames	Phase 1 model	Baseline	Candidate
Frames with a detection	77.3%	15.9%	21.7%
Total detections	9,665	1,913	3,049
Whole-frame boxes (≥ 80% of frame)	3,107	0	0



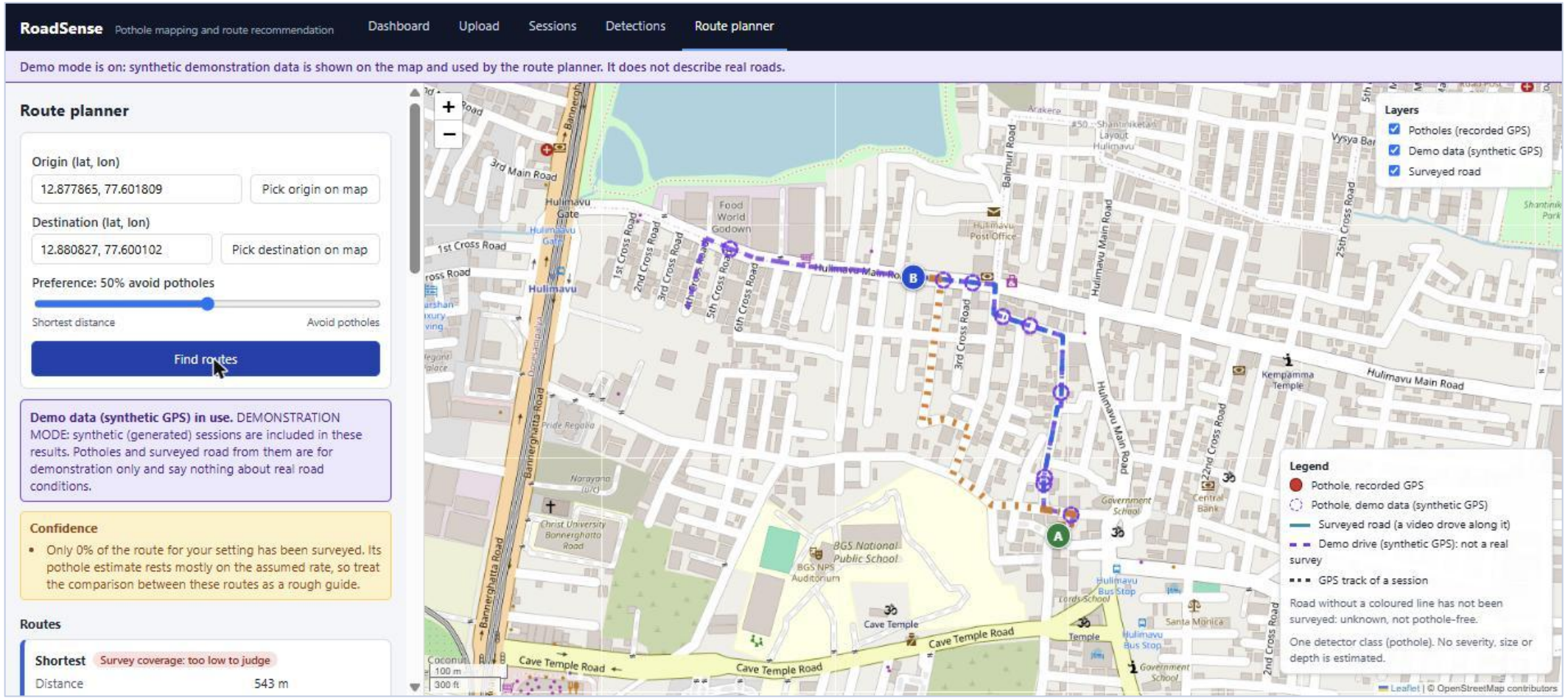
Same frame, three models: the Phase 1 model boxes the whole street as a pothole (0.87); both new models report nothing. No ground truth on our recordings, so this is a qualitative comparison.

Results: Snapshots of the Application



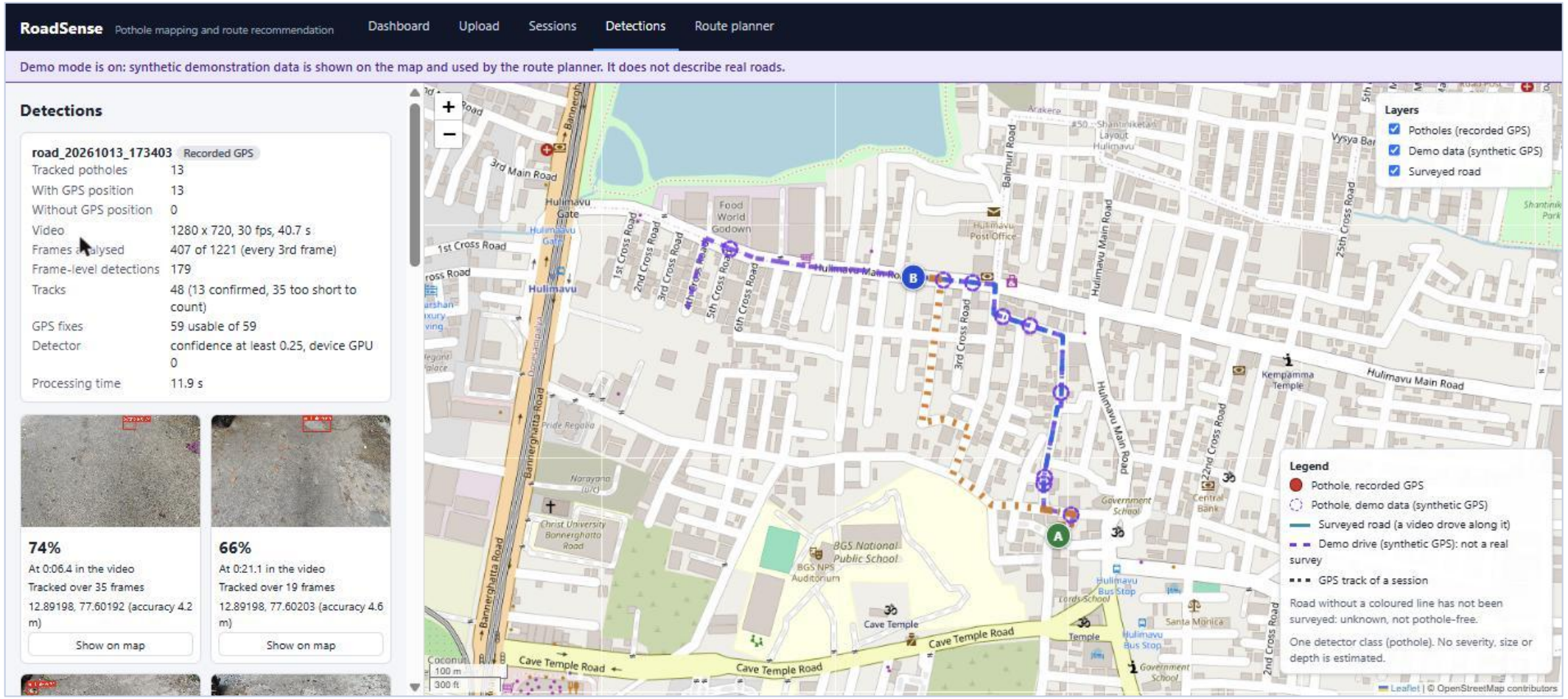
Dashboard: sessions, tracked sightings, mapped potholes and surveyed road length, with the map of recorded potholes (red) and the demo drive (purple).

Results: Snapshots of the Route Planner



Route planner: the shortest route (blue) and a pothole-avoiding route (orange), with a confidence note on how much of each route was surveyed. Shown on the synthetic demonstration session, because our real recordings cover only about 1 km.

Results: Snapshots of the Output



Detections for one real drive (road_20261013_173403): 13 tracked potholes from 407 analysed frames, each with its time in the video, frame count and GPS position.

Conclusion

- We rebuilt the system end to end: leakage-controlled data, a reproducible YOLOv8s baseline, a detection-to-map pipeline, pothole-aware routing, and a tested API with a web interface.
- The deployed baseline reaches mAP50 0.497 on held-out RDD2022 test data and runs at about 8 ms per frame on a laptop GPU.
- Fine-tuning at 1280 px fixes the baseline's blindness on forward-view dashcam footage (mAP50 0.018 → 0.454) but loses some recall on RDD2022, so it remains experimental.
- Road condition is reported as potholes per 100 m of surveyed road; route comparison is shown on a synthetic session.

0.497
baseline mAP50, RDD2022 test

25×
mAP50 gain on forward-view HRP4K

1,101
automated tests passing

SDG 3 · 9 · 11
goals addressed

Future Enhancements

1 Label our own roads

About 200 to 300 labelled frames from Bengaluru, including manholes and patches, to decide between the two models.

2 Fixed operating point

Declare a false-alarm budget in advance and confirm it once on data not yet seen.

3 Road condition scoring

Move from pothole counts to a calibrated condition score with severity or size.

4 Real route comparison

Record connected stretches of road so routes can be compared without synthetic data.

5 Security and UI tests

Add authentication to the API and automated tests for the web interface.

6 More forward-view data

Fine-tune on dashcam footage from Indian roads to improve recall where RDD2022 lost it.

Paper Published

Item	Details
Authors	Zain Mahedi, Thanu George, Vikas Shashi, Tanish Agrawal (guide: Dr. Seema Patil)
Title	AI Powered Pothole Detection and Intelligent Route Optimization System
Conference	2nd International Conference on Emerging Technologies in Computer Science for Interdisciplinary Applications (Track: Artificial Intelligence and Machine Learning)
Paper ID	722, submitted 30 September 2026 via Microsoft CMT
Status	Under review

References

[1] D. Arya, H. Maeda, S. K. Ghosh, D. Toshniwal and Y. Sekimoto, "RDD2022: A multi-national image dataset for automatic road damage detection," Geoscience Data Journal, vol. 11, pp. 846-862, 2024.

[2] H. Chen, Z. Tu, Y. Zhao and J. Ye, "A high-resolution perspective-view road image dataset for pothole detection," Scientific Data, vol. 13, Art. 961, 2026.

[3] E. Giordani, L. Arcioni, M. Gil-Martín, G. L. Foresti and M. R. Marini, "Real-world road damage dataset with potholes, cracks, and maintenance holes," Scientific Reports, vol. 16, Art. 15318, 2026.

[4] J. Redmon, S. Divvala, R. Girshick and A. Farhadi, "You only look once: Unified, real-time object detection," in Proc. IEEE CVPR, 2016, pp. 779-788.

[5] G. Jocher, A. Chaurasia and J. Qiu, Ultralytics YOLOv8, software, 2023. https://github.com/ultralytics/ultralytics

[6] J. Yosinski, J. Clune, Y. Bengio and H. Lipson, "How transferable are features in deep neural networks?" in Advances in NeurIPS, vol. 27, 2014, pp. 3320-3328.

[7] B. Barz and J. Denzler, "Do we train on test data? Purging CIFAR of near-duplicates," Journal of Imaging, vol. 6, no. 6, Art. 41, 2020.

[8] B. Efron and R. J. Tibshirani, An Introduction to the Bootstrap, Springer, 1993.

[9] R. Solovyev, W. Wang and T. Gabruseva, "Weighted boxes fusion: Ensembling boxes from different object detection models," Image and Vision Computing, vol. 107, Art. 104117, 2021.

[10] G. Boeing, "OSMnx: New methods for acquiring, constructing, analyzing, and visualizing complex street networks," Computers, Environment and Urban Systems, vol. 65, pp. 126-139, 2017.

[11] E. W. Dijkstra, "A note on two problems in connexion with graphs," Numerische Mathematik, vol. 1, pp. 269-271, 1959.

[12] Ministry of Road Transport and Highways, Road Accidents in India 2022, Government of India, New Delhi, 2023.

Thank You

Questions?